LAPORAN

WORKSHOP SISTEM INFORMASI WEB SERVER

(TIF330805)

MINGGU 9 PERTEMUAN 1

SEMESTER III

BKPM ACARA 17

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada praktikum ini dibuat API CRUD, yaitu layanan yang dapat menambah, menampilkan, mengubah, dan menghapus data melalui HTTP dan mengirim hasilnya dalam format JSON. API dibuat dengan PHP Native (tanpa framework) menggunakan pola MVC supaya kode lebih rapi.

Pada pola MVC, kode dibagi menurut tugasnya: Model mengurus database, Controller mengatur alur request dan response, dan Router menentukan Controller mana yang dipanggil berdasarkan URL. Seluruh request masuk lewat satu pintu, yaitu Front Controller (public/index.php).

2. Tujuan

1. Membuat struktur folder proyek berpola MVC.
2. Membuat Model (koneksi database dan query), Controller, Router, dan Front Controller.
3. Membuat API CRUD mahasiswa dengan response JSON.
4. Menguji API menggunakan Postman.

3. Dasar Teori Singkat

API CRUD. CRUD adalah empat operasi dasar pada data: Create (tambah), Read (tampil), Update (ubah), dan Delete (hapus). Pada API, setiap operasi memakai HTTP method yang berbeda.

Operasi	Fungsi	HTTP Method
Create	Menambahkan data baru	POST
Read	Menampilkan data	GET
Update	Mengubah data	PUT
Delete	Menghapus data	DELETE

Tabel 1. Operasi CRUD dan HTTP method

MVC. Model mengurus data dan database, View mengurus tampilan, dan Controller menerima request lalu menentukan Model yang dipakai dan response yang dikirim. View akan dipakai mulai acara 24.

Router dan Front Controller. Router membaca URL dan method HTTP lalu meneruskannya ke Controller yang sesuai. Front Controller adalah satu-satunya file yang diakses browser; file .htaccess mengarahkan semua request ke file tersebut.

PDO dan Prepared Statement. PDO adalah cara PHP mengakses database. Prepared statement memisahkan perintah SQL dari data yang dimasukkan pengguna sehingga aman dari SQL Injection.

Autoloader. spl_autoload_register() memuat file class secara otomatis, sehingga cukup menulis new NamaClass() tanpa require satu per satu.

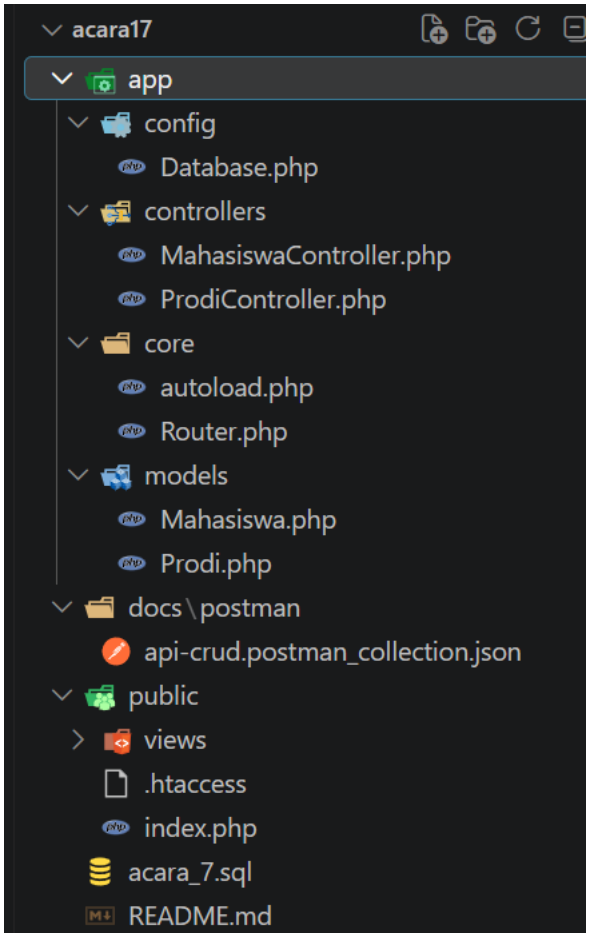
4. Alat dan Bahan

- XAMPP (Apache dengan mod_rewrite dan MySQL)
- Visual Studio Code
- Postman
- Web Browser
- File database acara_7.sql

HASIL PRAKTIK

1. Struktur Folder dan Database

Proyek dibuat dengan struktur MVC berikut. Folder app/ berisi kode aplikasi dan berada di luar public/, sehingga kode dan konfigurasi database tidak dapat dibuka langsung dari browser. Hanya folder public/ yang diakses browser.



Database yang dipakai adalah acara_7 (file acara_7.sql) yang sudah berisi tabel mahasiswa, prodi, dan matakuliah. Pada modul, tabel mahasiswa memakai kolom prodi berupa teks, sedangkan pada acara_7 program studi disimpan sebagai prodi_id yang berhubungan dengan tabel prodi. Karena itu body request memakai prodi_id, dan response menampilkan nama prodi dengan JOIN.

2. Lapisan Model

Class Database membuat satu koneksi PDO yang dipakai bersama oleh semua Model. Jika koneksi gagal, API menjawab 500 dengan pesan umum.

```
// Model: koneksi database dengan PDO (satu koneksi dipakai bersama / Singleton).
class Database {
    private static ?PDO $connection = null;
```

```
    public static function getConnection(): PDO {
        if (self::$connection === null) {
            $host    = 'localhost';
            $db      = 'acara_7';
            $charset = 'utf8mb4';

            $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
            $options = [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ];

            try {
                self::$connection = new PDO($dsn, 'root', '', $options);
            } catch (PDOException $e) {
                http_response_code(500);
                header('Content-Type: application/json');
                echo json_encode(['success' => false, 'message' => 'Koneksi database
gagal']);
                exit;
            }
        }

        return self::$connection;
    }
}
```

Gambar 3. app/config/Database.php

Class Mahasiswa menyimpan seluruh query SQL untuk tabel mahasiswa dengan prepared statement. Method getAll() dan find() menampilkan data beserta nama prodi, sedangkan create(), update(), dan delete() mengubah data. Method update() dan delete() ada di file yang sama.

```
class Mahasiswa {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    // Seluruh mahasiswa, lengkap dengan nama prodi (JOIN)
    public function getAll(): array {
        return $this->db->query(
            'SELECT m.*, p.nama AS prodi
             FROM mahasiswa m JOIN prodi p ON m.prodi_id = p.id
             ORDER BY m.id DESC'
        )->fetchAll();
    }

    public function find(int $id) {
        $stmt = $this->db->prepare(
            'SELECT m.*, p.nama AS prodi
             FROM mahasiswa m JOIN prodi p ON m.prodi_id = p.id
             WHERE m.id = :id'
        );
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }
```

```
    public function create(array $data): string {
        $this->validate($data);

        $stmt = $this->db->prepare(
            'INSERT INTO mahasiswa (nim, nama, prodi_id, angkatan, status)
             VALUES (:nim, :nama, :prodi_id, :angkatan, :status)'
        );
        $stmt->execute([
            ':nim'      => $data['nim'],
            ':nama'     => $data['nama'],
            ':prodi_id' => $data['prodi_id'],
            ':angkatan' => $data['angkatan'],
            ':status'   => $data['status'],
        ]);
        return $this->db->lastInsertId();
    }
```

3. Lapisan Controller

MahasiswaController tidak menulis SQL. Ia hanya memanggil method Model, lalu mengirim response JSON beserta status code: 200 (berhasil), 201 (data dibuat), 400 (data tidak valid), dan 404 (data tidak ditemukan). Berikut method index() untuk GET dan store() untuk POST. Method update() dan destroy() memakai pola yang sama.

```
class MahasiswaController {
    private Mahasiswa $model;

    public function __construct() {
        $this->model = new Mahasiswa();
    }

    // GET /mahasiswa  atau  GET /mahasiswa?id=1
    public function index(): void {
        header('Content-Type: application/json');

        $id = $_GET['id'] ?? null;

        if ($id !== null) {
            $data = ctype_digit((string) $id) ? $this->model->find((int) $id) :
false;

            if (!$data) {
                http_response_code(404);
                echo json_encode(['success' => false, 'message' => 'Data tidak
ditemukan']);
                exit;
            }

            echo json_encode(['success' => true, 'data' => $data]);
            exit;
        }

        echo json_encode(['success' => true, 'data' => $this->model->getAll()]);
    }

    // POST /mahasiswa
    public function store(): void {
```

```
        header('Content-Type: application/json');

        $data  = json_decode(file_get_contents('php://input'), true);
        $error = $this->validasi($data);

        if ($error) {
            http_response_code(400);
            echo json_encode(['success' => false, 'message' => $error]);
            exit;
        }

        $data['status'] = $data['status'] ?? 'aktif';

        try {
            $id = $this->model->create($data);
        } catch (InvalidArgumentException $e) {
            http_response_code(400);
            echo json_encode(['success' => false, 'message' => $e->getMessage()]);
            exit;
        }

        http_response_code(201);
        echo json_encode([
            'success' => true,
            'message' => 'Data berhasil ditambahkan',
            'data'    => ['id' => $id],
        ]);
    }
```

4. Autoloader, Router, dan Front Controller

Autoloader memuat class dari folder config, core, models, dan controllers secara otomatis.

```
// Memuat file class secara otomatis, sehingga cukup menulis new NamaClass()
// tanpa require satu per satu.
spl_autoload_register(function ($class) {
    $folders = ['config', 'core', 'models', 'controllers'];

    foreach ($folders as $folder) {
        $path = __DIR__ . "/../{$folder}/{$class}.php";
        if (file_exists($path)) {
            require $path;
            return;
        }
    }
});
```

Router mengambil nama resource dari URL (misalnya mahasiswa), mencocokkannya dengan peta Controller, lalu memanggil method Controller sesuai HTTP method: GET ke index(), POST ke store(), PUT ke update(), dan DELETE ke destroy(). Resource yang tidak ada dijawab 404.

```
// Router: membaca URL dan method HTTP, lalu meneruskannya ke Controller.
class Router {
    // Peta sederhana: nama resource pada URL => nama class Controller
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'prodi'     => 'ProdiController',
    ];
```

```
    public function dispatch(): void {
        $uri      = trim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/');
        $segments = explode('/', $uri);
        $resource = end($segments) ?: 'mahasiswa';

        $controllerName = $this->map[$resource] ?? null;

        if (!$controllerName) {
            http_response_code(404);
            header('Content-Type: application/json');
            echo json_encode(['success' => false, 'message' => 'Endpoint tidak
ditemukan']);
            exit;
        }

        $controller = new $controllerName();

        try {
            switch ($_SERVER['REQUEST_METHOD']) {
                case 'GET':    $controller->index(); break;
                case 'POST':   $controller->store(); break;
                case 'PUT':    $controller->update($_GET['id'] ?? null); break;
                case 'DELETE': $controller->destroy($_GET['id'] ?? null); break;
                default:
                    http_response_code(405);
                    header('Content-Type: application/json');
                    echo json_encode(['success' => false, 'message' => 'Method tidak
diizinkan']);
            }
        } catch (PDOException $e) {
            // Detail error database hanya dicatat, tidak dikirim ke client.
            error_log($e->getMessage());
            http_response_code(500);
            header('Content-Type: application/json');
            echo json_encode(['success' => false, 'message' => 'Terjadi kesalahan
pada server']);
        }
    }
}
```

Front Controller adalah pintu masuk tunggal, dan .htaccess mengarahkan semua request yang bukan file atau folder ke index.php.

```
// Front Controller: satu-satunya pintu masuk. Semua request masuk ke sini,
// lalu diteruskan ke Router.
require_once __DIR__ . '/../app/core/autoload.php';

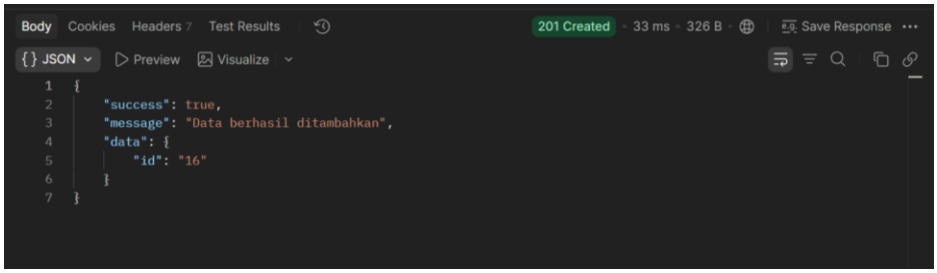
(new Router())->dispatch();
```

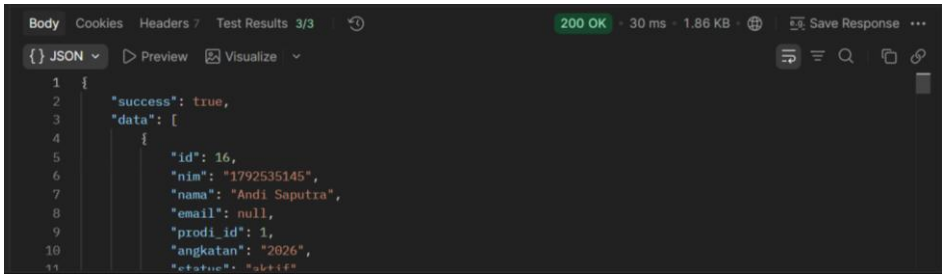
```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

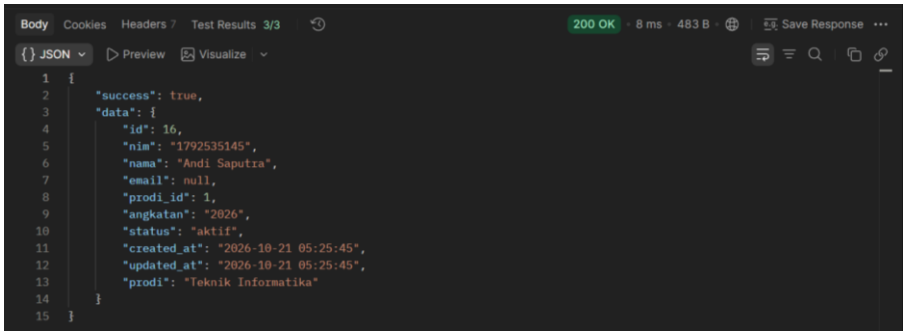
5. Pengujian dengan Postman

Pengujian dilakukan di Postman ke alamat http://localhost/sistem-informasi-akademik/public/mahasiswa. Alur setiap request: Router meneruskan ke Controller, lalu Controller memanggil method Model yang sesuai.

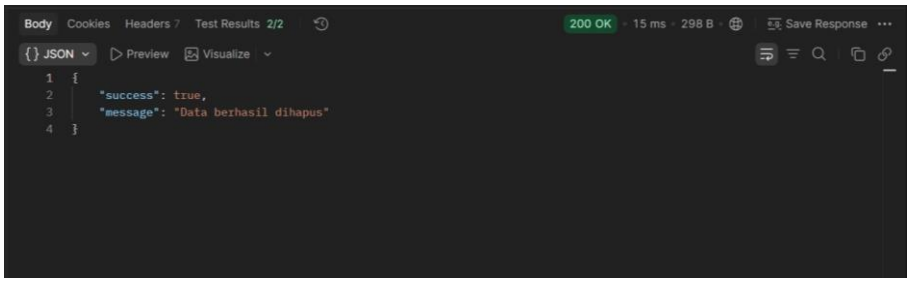
No	Request	Status	Hasil yang diharapkan
1	POST /mahasiswa (body JSON)	201	Data baru tersimpan, response berisi id
2	GET /mahasiswa	200	Menampilkan seluruh mahasiswa
3	GET /mahasiswa?id=1	200	Menampilkan satu mahasiswa
4	PUT /mahasiswa?id=1 (body JSON)	200	Data berhasil diperbarui
5	DELETE /mahasiswa?id=1	200	Data berhasil dihapus
6	GET /mahasiswa?id=1 (setelah dihapus)	404	Data tidak ditemukan







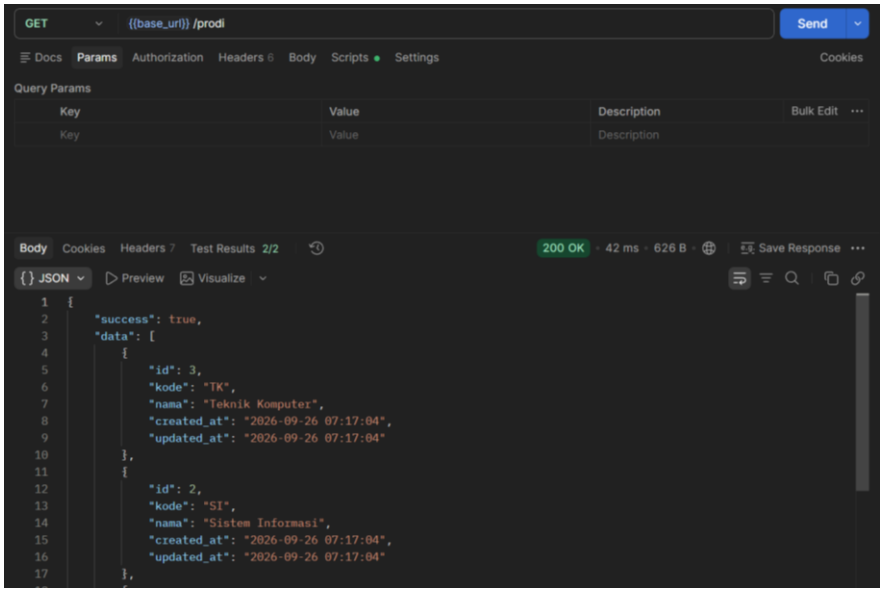






6. Tugas Praktik Mandiri

Tugas 1: resource lain. Dibuat Model Prodi dan ProdiController dengan pola yang sama seperti Mahasiswa, lalu resource prodi didaftarkan pada peta Router (baris 'prodi' => 'ProdiController'). Resource ini memakai tabel prodi yang sudah ada pada acara_7, sehingga tidak ada tabel baru. Prodi yang masih dipakai mahasiswa tidak dapat dihapus (dijawab 400).

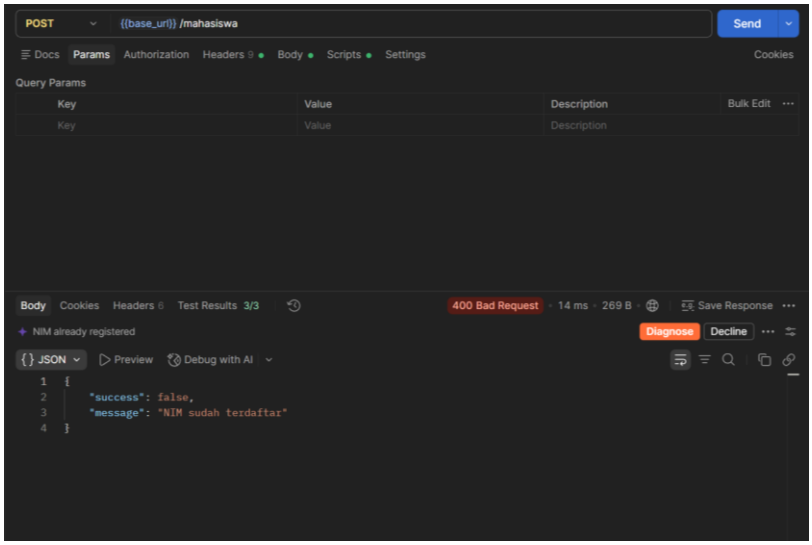


Tugas 2: validasi NIM duplikat di Model. Method validate() pada Model Mahasiswa memeriksa apakah NIM sudah dipakai sebelum data disimpan. Jika sudah, Model melempar pesan "NIM sudah terdaftar" dan Controller menjawab 400. Saat mengubah data, NIM milik data itu sendiri tidak dianggap duplikat. Model ini juga memeriksa bahwa prodi_id yang dikirim ada pada tabel prodi.

```
    // Validasi di Model (Tugas Mandiri 2): NIM tidak boleh duplikat dan
    // program studi harus ada. $exceptId dipakai saat update agar NIM milik
    // data itu sendiri tidak dianggap duplikat.
    private function validate(array $data, ?int $exceptId = null): void {
        $sql    = 'SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim';
        $params = [':nim' => $data['nim']];
        if ($exceptId !== null) {
            $sql .= ' AND id <> :id';
            $params[':id'] = $exceptId;
        }
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);
        if ((int) $stmt->fetchColumn() > 0) {
            throw new InvalidArgumentException('NIM sudah terdaftar');
        }

        $stmt = $this->db->prepare('SELECT COUNT(*) FROM prodi WHERE id = :id');
        $stmt->execute([':id' => $data['prodi_id']]);
        if ((int) $stmt->fetchColumn() === 0) {
            throw new InvalidArgumentException('Program studi tidak ditemukan');
        }
    }
}
```

```
{
    "success": false,
    "message": "NIM sudah terdaftar"
}
```



Tugas 3: dokumentasi endpoint dan diagram alur. Daftar endpoint, serta diagram alur Router, Controller, dan Model:

Method	Alamat	Fungsi	Status
GET	/public/mahasiswa	Seluruh mahasiswa	200
GET	/public/mahasiswa?id=1	Satu mahasiswa	200, 404
POST	/public/mahasiswa	Tambah mahasiswa	201, 400
PUT	/public/mahasiswa?id=1	Ubah mahasiswa	200, 400, 404
DELETE	/public/mahasiswa?id=1	Hapus mahasiswa	200, 404
GET/POST/PUT/DELETE	/public/prodi	CRUD program studi (sama polanya)	200, 201, 400, 404

Contoh request dan response (berdasarkan data awal acara_7):

POST /mahasiswa, body request dan response:

```
{
    "nim": "2401010",
    "nama": "Andi Saputra",
    "prodi_id": 1,
    "angkatan": 2026,
    "status": "aktif"
}
```

```
{
    "success": true,
    "message": "Data berhasil ditambahkan",
    "data": {
        "id": "5"
    }
}
```

GET /mahasiswa?id=1:

```
{
    "success": true,
    "data": {
        "id": 1,
        "nim": "2401001",
        "nama": "Budi Santoso",
        "prodi_id": 1,
        "angkatan": 2024,
        "status": "aktif",
        "created_at": "2026-09-26 00:17:04",
        "updated_at": "2026-09-26 00:17:04",
        "prodi": "Teknik Informatika"
    }
}
```

PUT /mahasiswa?id=1 (body seperti POST) dan DELETE /mahasiswa?id=1:

```
{
    "success": true,
    "message": "Data berhasil diperbarui"
}
```

)

```
{
    "success": true,
    "message": "Data berhasil dihapus"
}
```

```
{
    "success": false,
    "message": "Data tidak ditemukan"
}
```

7. Pembahasan

- Kode dibagi menjadi Model (query), Controller (alur dan response), dan Router (pemilihan Controller). Karena itu perubahan pada satu bagian tidak mengacaukan bagian lain.
- Query SQL hanya ada di Model dan memakai prepared statement, sehingga masukan pengguna tidak dapat mengubah perintah SQL.
- Front Controller membuat semua request lewat satu pintu, sehingga alamat API bersih tanpa .php dan folder app/ tidak terbuka dari browser.
- Menambah resource baru (seperti prodi) cukup membuat Model dan Controller lalu mendaftarkannya di Router.

KESIMPULAN

1. Struktur proyek berpola MVC berhasil dibuat dengan Router dan Front Controller sebagai satu-satunya pintu masuk.
2. API CRUD mahasiswa berhasil dibuat dengan PHP Native, PDO, dan prepared statement, dan response berformat JSON.
3. Seluruh endpoint (POST, GET, PUT, DELETE) berhasil diuji menggunakan Postman.
4. Tugas mandiri berhasil dikerjakan: resource prodi, validasi NIM duplikat pada Model, serta dokumentasi endpoint dan diagram alur.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.